Aula 07 — Persistência: banco de dados com SQLite

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

> **Duração:** 2h · **Pré-requisito:** Aula 06 (API refatorada em camadas)

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar por que dados em memória não servem para um sistema real.
2. Entender o que é um banco relacional: tabelas, colunas, linhas, chaves.
3. Escrever os comandos SQL básicos: `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`.
4. Conectar a API ao SQLite usando o módulo nativo `node:sqlite`.
5. Escrever consultas seguras com **parâmetros**, evitando SQL Injection.

> **Nota de ambiente:** usamos o módulo **nativo** `node:sqlite`, que já vem com o Node. Nenhuma instalação, nenhum servidor de banco, nenhum problema de compilação em 20 máquinas diferentes. Requer **Node 22.5 ou superior**; no Node 22 é preciso a flag `--experimental-sqlite` (no Node 24+ ela é desnecessária, mas inofensiva).

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Conferir a refatoração da aula 06 — quem não fez, pega a solução
0:10–0:35	Conceito	Por que banco · tabelas · SQL básico na prática
0:35–1:05	Código ao vivo	Criar o banco, o schema e **trocar só o repository**
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 4
1:50–2:00	Fechamento	SQL Injection · prévia: começa o front-end

---

1. O problema dos dados em memória

Faça o teste em sala: crie uma tarefa pela API, reinicie o servidor, liste de novo. Sumiu.

```
let tarefas = [ ... ];   // isso vive na RAM
```

Quando o processo Node morre — restart, deploy, queda de energia — a RAM é liberada e tudo se perde. Nenhum sistema real funciona assim.

Precisamos de algo que:

Requisito	Por quê
Sobreviva ao restart	dados são o ativo do sistema
Busque rápido em muitos dados	`array.find()` em 1 milhão de itens é inviável
Aceite vários acessos ao mesmo tempo	um servidor atende muita gente
Garanta integridade	não pode existir pedido sem cliente

Isso é um **banco de dados**.

---

2. Bancos relacionais em 5 minutos

Um banco relacional guarda dados em **tabelas** — na prática, planilhas com regras:

```
                  TABELA: tarefas

┌────┬──────────────────────┬────────────┬──────────────┐
│ id │ titulo               │ concluida  │ prioridade   │  ← COLUNAS (campos)
├────┼──────────────────────┼────────────┼──────────────┤
│ 1  │ Estudar Node.js      │ 1          │ alta         │  ← LINHA (registro)
│ 2  │ Aprender Express     │ 0          │ alta         │
│ 3  │ Entender SQL         │ 0          │ media        │
└────┴──────────────────────┴────────────┴──────────────┘
```

Conceitos essenciais:

- **Tabela** = uma coleção de coisas do mesmo tipo (tarefas, usuários, produtos).
- **Coluna** = um campo, com **tipo definido** (texto, número, data).
- **Linha** = um registro.
- **Chave primária (PK)** = a coluna que identifica cada linha de forma única. Sempre o `id`.
- **Chave estrangeira (FK)** = uma coluna que aponta para o `id` de outra tabela. É a "relação" que dá nome a "banco relacional".

Por que SQLite

	SQLite	PostgreSQL / MySQL
Instalação	nenhuma — é um arquivo	precisa instalar e configurar servidor
Onde vive	`dados.db` na sua pasta	processo separado, com porta e senha
Bom para	aprender, apps pequenos, mobile	sistemas em produção com muita gente

O **SQL é praticamente o mesmo** nos três. Aprender SQLite hoje é aprender PostgreSQL amanhã — só muda a conexão. Por isso começamos aqui.

---

3. SQL: os cinco comandos que importam

Criar a tabela

```sql
CREATE TABLE IF NOT EXISTS tarefas (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  titulo      TEXT    NOT NULL,
  concluida   INTEGER NOT NULL DEFAULT 0,
  prioridade  TEXT    NOT NULL DEFAULT 'media',
  criada_em   TEXT    NOT NULL
);
```

Traduzindo as palavras-chave:

Palavra	Significa
`PRIMARY KEY`	identifica a linha; não pode repetir
`AUTOINCREMENT`	o banco gera o número sozinho
`NOT NULL`	esta coluna é obrigatória
`DEFAULT x`	se ninguém informar, vale `x`

⚠️ **O SQLite não tem tipo booleano.** Use `INTEGER` com `0` (falso) e `1` (verdadeiro), e converta na fronteira do repository. Datas também são guardadas como `TEXT`, no formato ISO.

Inserir

```sql
INSERT INTO tarefas (titulo, concluida, prioridade, criada_em)
VALUES ('Estudar SQL', 0, 'alta', '2026-04-01T10:00:00.000Z');
```

Buscar

```sql
SELECT * FROM tarefas;                                  -- tudo
SELECT id, titulo FROM tarefas;                         -- só algumas colunas
SELECT * FROM tarefas WHERE concluida = 0;              -- com filtro
SELECT * FROM tarefas WHERE titulo LIKE '%node%';       -- contém "node"
SELECT * FROM tarefas ORDER BY criada_em DESC;          -- ordenado
SELECT * FROM tarefas LIMIT 10;                         -- no máximo 10
SELECT COUNT(*) AS total FROM tarefas WHERE concluida = 1;  -- contar
```

Repare como isso é o mesmo `filter`, `sort` e `length` da aula 02 — só que executado pelo banco, que faz isso muito mais rápido em grande volume.

Atualizar e apagar

```sql
UPDATE tarefas SET concluida = 1 WHERE id = 3;
DELETE FROM tarefas WHERE id = 3;
```

🚨 **O erro mais caro do SQL:** esquecer o `WHERE`. `UPDATE tarefas SET concluida = 1;` marca **todas** as tarefas como concluídas. `DELETE FROM tarefas;` apaga **tudo**. Um dia você vai fazer isso. Que seja num banco de teste. Escreva o `WHERE` **primeiro**, depois volte e escreva o começo do comando.

---

4. Usando SQLite no Node

```
import { DatabaseSync } from "node:sqlite";

const db = new DatabaseSync("./dados.db");   // cria o arquivo se não existir

// exec: para comandos sem retorno (criar tabelas)
db.exec(`CREATE TABLE IF NOT EXISTS tarefas (...)`);

// prepare + all: várias linhas
const tarefas = db.prepare("SELECT * FROM tarefas").all();

// prepare + get: uma linha (ou undefined)
const tarefa = db.prepare("SELECT * FROM tarefas WHERE id = ?").get(3);

// prepare + run: alterações — devolve { changes, lastInsertRowid }
const resultado = db.prepare("INSERT INTO tarefas (titulo) VALUES (?)").run("Nova");
console.log(resultado.lastInsertRowid);   // o id gerado pelo banco
console.log(resultado.changes);            // quantas linhas mudaram
```

Três métodos, três usos:

Método	Use quando	Devolve
`.all()`	espera várias linhas	array (vazio se nada bater)
`.get()`	espera uma linha	objeto ou `undefined`
`.run()`	INSERT / UPDATE / DELETE	`{ changes, lastInsertRowid }`

`changes` é ouro: `changes === 0` num `UPDATE` significa "não existe linha com esse id" — é assim que o repository sabe devolver `null` e o service lançar 404.

---

5. 🚨 SQL Injection — a parte séria da aula

Nunca, jamais, em hipótese alguma monte SQL com template literal:

```
// ❌❌❌ NUNCA FAÇA ISSO
const busca = req.query.busca;
db.prepare(`SELECT * FROM tarefas WHERE titulo = '${busca}'`).all();
```

Se alguém enviar `busca = "'; DROP TABLE tarefas; --"`, o comando executado vira:

```
SELECT * FROM tarefas WHERE titulo = ''; DROP TABLE tarefas; --'
```

Sua tabela some. Essa falha, chamada **SQL Injection**, é uma das mais antigas e ainda uma das mais exploradas do mundo — e já derrubou empresas inteiras.

A forma correta é usar parâmetros (`?`):

```
// ✅ SEMPRE ASSIM
db.prepare("SELECT * FROM tarefas WHERE titulo = ?").all(busca);
```

Com `?`, o banco trata o valor **como dado**, nunca como comando. Mesmo que o texto contenha `DROP TABLE`, ele será apenas um título esquisito.

> **Regra sem exceção:** todo valor que vem de fora entra por `?`. Sem exceção, nem em código de teste, nem "só dessa vez".

---

6. Trocando o repository — e só ele

Aqui está a recompensa da aula 06. O `service`, o `controller` e as `routes` **não mudam uma linha**. O contrato continua o mesmo: `buscarPorId(id)` devolve uma tarefa ou `null`.

```
// repositories/tarefa.repository.js — versão SQLite
import { db } from "../banco/conexao.js";

// O banco fala "snake_case" (criada_em); o JavaScript fala "camelCase" (criadaEm).
// Traduzir entre os dois é trabalho DESTA camada — e de mais ninguém.
function paraObjeto(linha) {
  if (!linha) return null;
  return {
    id: linha.id,
    titulo: linha.titulo,
    concluida: linha.concluida === 1,   // INTEGER → boolean
    prioridade: linha.prioridade,
    criadaEm: linha.criada_em,
  };
}

export const tarefaRepository = {
  listarTodas() {
    return db.prepare("SELECT * FROM tarefas ORDER BY id").all().map(paraObjeto);
  },

  buscarPorId(id) {
    return paraObjeto(db.prepare("SELECT * FROM tarefas WHERE id = ?").get(id));
  },

  criar(dados) {
    const resultado = db
      .prepare("INSERT INTO tarefas (titulo, concluida, prioridade, criada_em) VALUES (?,
        ?, ?, ?)")
      .run(dados.titulo, dados.concluida ? 1 : 0, dados.prioridade, dados.criadaEm);

    return this.buscarPorId(resultado.lastInsertRowid);
  },

  remover(id) {
    return db.prepare("DELETE FROM tarefas WHERE id = ?").run(id).changes > 0;
  },
};
```

Faça esse momento render em sala. Mostre o `git diff`: um arquivo alterado, o resto intocado. É a prova concreta de que a arquitetura da aula 06 valeu a pena — e a melhor forma de convencer quem achou que era burocracia.

7. Organização do banco no projeto

```
src/
├── banco/
│   ├── conexao.js     ← abre o arquivo .db e exporta a conexão
│   ├── schema.sql     ← o CREATE TABLE
│   └── popular.js     ← script que insere dados de exemplo (seed)
└── repositories/
    └── tarefa.repository.js
```

```js
// banco/conexao.js
import { DatabaseSync } from "node:sqlite";
import { readFileSync } from "node:fs";

const CAMINHO = process.env.BANCO_ARQUIVO ?? "./dados.db";

export const db = new DatabaseSync(CAMINHO);

// Liga a verificação de chaves estrangeiras — no SQLite ela vem DESLIGADA por padrão.
db.exec("PRAGMA foreign_keys = ON");

// Roda o schema na inicialização. Com IF NOT EXISTS, rodar de novo não faz mal.
db.exec(readFileSync(new URL("./schema.sql", import.meta.url), "utf-8"));
```

E no `.gitignore`:

```
*.db
```

O banco é **dado**, não código. Cada pessoa tem o seu. O que vai para o Git é o `schema.sql` (a estrutura) e o `popular.js` (os dados de exemplo).

8. Relacionamentos (introdução)

Uma tarefa pertence a um usuário:

```sql
CREATE TABLE usuarios (
  id   INTEGER PRIMARY KEY AUTOINCREMENT,
  nome TEXT NOT NULL
);

CREATE TABLE tarefas (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  titulo     TEXT NOT NULL,
  usuario_id INTEGER NOT NULL,
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE
);
```

`FOREIGN KEY` diz ao banco: *"o valor de `usuario_id` precisa existir na tabela `usuarios`"*. Isso impede tarefa órfã — o banco **recusa** o insert. `ON DELETE CASCADE` completa: apagou o usuário, as tarefas dele vão junto.

Para buscar dados das duas tabelas de uma vez, usa-se `JOIN`:

```sql
SELECT tarefas.titulo, usuarios.nome
FROM tarefas
JOIN usuarios ON usuarios.id = tarefas.usuario_id;
```

`JOIN` é o assunto de um módulo inteiro. Hoje basta saber que ele existe e para que serve.

---

9. Ferramentas novas desta aula

`DatabaseSync` e os três métodos

```js
import { DatabaseSync } from "node:sqlite";

const db = new DatabaseSync("./dados.db");   // cria o arquivo se não existir
```

Método	Use quando	Devolve
`db.exec(sql)`	comandos sem retorno (criar tabela, `BEGIN`)	nada
`db.prepare(sql).all(...)`	espera **várias** linhas	array (vazio se nada bater)
`db.prepare(sql).get(...)`	espera **uma** linha	objeto ou `undefined`
`db.prepare(sql).run(...)`	`INSERT`, `UPDATE`, `DELETE`	`{ changes, lastInsertRowid }`

Por que `prepare` separado? Porque a consulta é "preparada" uma vez e pode ser executada várias, com valores diferentes — é mais rápido e, principalmente, é o que permite usar `?` com segurança.

```
const inserir = db.prepare("INSERT INTO tarefas (titulo) VALUES (?)");
for (const t of lista) inserir.run(t.titulo);   // prepara 1x, executa N vezes
```

`changes` e `lastInsertRowid`

O objeto devolvido por `.run()` tem duas informações valiosas:

```
const resultado = db.prepare("UPDATE tarefas SET titulo = ? WHERE id = ?").run("Novo", 5);

resultado.changes;          // quantas linhas foram afetadas
resultado.lastInsertRowid;  // o id gerado pelo AUTOINCREMENT (em INSERTs)
```

`changes === 0` num `UPDATE` ou `DELETE` significa **"não existe linha com esse id"**. É assim que o repository sabe devolver `null` e o service sabe lançar o 404:

```
return resultado.changes === 0 ? null : this.buscarPorId(id);
```

O `?` — parâmetros, não concatenação

```
db.prepare("SELECT * FROM tarefas WHERE titulo = ?").all(busca);
//                                                ↑        ↑
//                              o espaço reservado    o valor
```

Cada `?` é preenchido, **na ordem**, pelos argumentos passados. O banco trata esses valores como **dado**, nunca como comando — e é isso que impede o SQL Injection.

Com vários parâmetros, use `.run(...valores)` para espalhar um array:

```
db.prepare(`UPDATE tarefas SET ${colunas.join(", ")} WHERE id = ?`).run(...valores);
```

⚠️ Repare na distinção crucial: os **nomes de coluna** vêm do seu código (por isso podem ser concatenados). Os **valores** vêm de fora e vão obrigatoriamente por `?`.

`Math.max(...)` com spread

```
Math.max(3, 7, 2);                          // 7
Math.max([3, 7, 2]);                        // NaN 😴 — não aceita array
Math.max(...[3, 7, 2]);                     // 7   ✅
Math.max(...lista.map((item) => item.id));  // o maior id da lista
```

`Math.max` recebe números soltos, não um array. Os três pontinhos "espalham" o array em números soltos. É assim que geramos o próximo id quando o banco não faz isso por nós.

`Array.isArray(valor)`

```
Array.isArray([1, 2]);      // true
Array.isArray("texto");     // false
typeof [];                  // "object"  😴 — não ajuda em nada
```

Por que existe: `typeof` não distingue array de objeto. Quando você precisa garantir que o corpo da requisição é mesmo uma lista (na importação em lote, por exemplo), `Array.isArray` é a forma correta.

`.flatMap()`

```
[1, 2].flatMap((n) => [n, n * 10]);    // [1, 10, 2, 20]
```

É um `.map()` seguido de "achatar" um nível. Usamos para juntar as listas de erros de vários itens numa lista única:

```
errosPorItem.flatMap((item) => item.erros.map((erro) => `filme ${item.indice}: ${erro}`));
```

Sem o `flatMap`, o resultado seria um array de arrays.

`PRAGMA foreign_keys = ON`

```
db.exec("PRAGMA foreign_keys = ON");
```

`PRAGMA` é um comando de **configuração** do SQLite. Este liga a verificação de chaves estrangeiras — que, por motivos históricos, vem **desligada por padrão**.

Sem essa linha, o banco aceita silenciosamente uma avaliação apontando para um filme que não existe, e o `ON DELETE CASCADE` não funciona. É uma linha fácil de esquecer e difícil de descobrir depois.

Transações: `BEGIN`, `COMMIT`, `ROLLBACK`

```
db.exec("BEGIN TRANSACTION");
try {
  // várias operações...
  db.exec("COMMIT");     // confirma todas de uma vez
} catch (erro) {
  db.exec("ROLLBACK");   // desfaz TODAS as operações desde o BEGIN
  throw erro;
}
```

Uma transação garante **tudo ou nada**. É o que impede o banco de ficar num estado pela metade quando a terceira de cinco inserções falha.

Recapitulação em 3 frases

1. Dados em memória somem no restart; banco de dados é o que sobrevive.
2. Cinco comandos resolvem quase tudo: `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE` — e **nunca esqueça o `WHERE`**.
3. Todo valor vindo de fora entra por `?`; montar SQL com template literal é SQL Injection.

Tarefa para a próxima aula

Exercícios 1 a 4 de `exercicios/README.md`.

Na próxima aula

O back-end está pronto. Agora vamos construir a **tela**: começa o React.
